
pdf_options:
format: a4
margin: 20mm

```
<style>
body {
font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
line-height: 1.6;
color: #333;
}
h1 { color: #2c3e50; text-align: center; border-bottom: 2px solid #eee; padding-bottom: 10px; }
h2 { color: #34495e; margin-top: 30px; border-bottom: 1px solid #eee; padding-bottom: 5px; }
h3 { color: #7f8c8d; }
code { background-color: #f8f9fa; padding: 2px 4px; border-radius: 4px; font-size: 0.9em; }
pre { background-color: #f8f9fa; padding: 15px; border-radius: 8px; border: 1px solid #e9ecef; }
.formula { font-size: 1.2em; text-align: center; margin: 20px 0; color: #2c3e50; font-family: "Georgia", serif; }
</style>
```

Анализ отказоустойчивости спутниковой группировки: Модель Монте-Карло

В данном документе описывается математическая модель и техническая реализация модуля симуляции по методу Монте-Карло, разработанного для оценки надежности и отказоустойчивости низкоорбитальной спутниковой группировки.

1. Введение и цели моделирования

Проектирование спутниковой группировки требует учета вероятностных факторов, таких как внезапный выход космических аппаратов (КА) из строя. Классические детерминированные тесты (например, отказ одного конкретного спутника) не дают полной картины риска.

Метод Монте-Карло в нашем решении применяется для:

- Вероятностной оценки SLA** (Service Level Agreement) — насколько часто реальная доступность сети будет падать ниже целевых 90%.
 - Оценки финансового риска** — расчета математического ожидания штрафов и затрат на экстренные запуски замещающих КА.
 - Расчета TCO** (Total Cost of Ownership) с учетом нештатных ситуаций, а не только идеальных условий.
-

2. Математическая модель

2.1. Биномиальное распределение отказов

Предположим, что группировка на текущем этапе развертывания состоит из N активных спутников. Вероятность выхода из строя одного спутника в течение заданного горизонта планирования (например, 1 день) составляет p .

Поскольку отказы отдельных аппаратов в базовом сценарии считаются независимыми событиями, вероятность того, что из строя выйдут ровно k спутников, описывается формулой биномиального распределения:

$$P(X=k) = C_{N-k}^k \cdot p^k \cdot (1-p)^{N-k}$$

где $C_{N-k}^k = \frac{N!}{k!(N-k)!}$ — число сочетаний.

2.2. Стратифицированное сэмплирование (Stratified Sampling)

При случайной генерации событий (наивный Монте-Карло) с малой вероятностью $p = 0.01$ мы почти всегда будем получать $k=0$ или $k=1$. Однако для оценки рисков критически важно исследовать редкие, но разрушительные события (например, $k=5$).

Для этого алгоритм искусственно генерирует сценарии для всех возможных k (от 0 до N), но на этапе агрегации **взвешивает** каждый результат на реальную вероятность $P(X=k)$.

2.3. Расчет Ожидаемого Финансового Риска (Expected Risk)

Для каждого сценария i с отказами рассчитывается общая стоимость ликвидации последствий и штрафов ($Cost_i$).

Математическое ожидание (Expected Risk) вычисляется как сумма произведений вероятностей на ущерб:

$$E[Risk] = \sum (P_i \times Cost_i)$$

Ущерб ($Cost_i$) состоит из:

- **Штрафов за нарушение SLA:** зависят от количества отключенных клиентов и длительности простоя.
- **Стоимости экстренных запусков:** если выходит из строя более 15% группировки, требуется запуск замещающей ракеты (включает стоимость ракеты и новых КА).

2.4. Совокупная стоимость владения (TCO)

В отличие от классического подхода (CAPEX + OPEX), наша модель оценивает реальную стоимость с учетом рисков:

$$TCO = CAPEX + OPEX + E[Risk]$$

3. Техническая реализация (Архитектура)

Программный модуль состоит из трех ключевых слоев, обеспечивающих высокую производительность и защиту от взаимных блокировок.

3.1. Генератор сценариев (scenario_generator.py)

Генератор принимает базовую JSON-модель группировки и создает десятки и сотни производных сценариев:

- Рассчитывает количество выборок ($n_samples$) для каждого количества отказов k .
- Вносит случайные точки отказа (время старта и ID аппарата) в сценарий.
- Добавляет коррелированные сценарии (солнечные вспышки, каскадный синдром Кесслера).
- Присваивает каждому сгенерированному объекту его математическую вероятность ($_mc_prob$).

3.2. Изолированный пул процессов (parallel_runner.py)

Поскольку топологический анализ графа связи для 50+ узлов каждую минуту является процессороемкой задачей, симуляции выполняются параллельно.

- Внедрен **глобальный пул процессов** (`ProcessPoolExecutor(max_workers=CPU-1)`).
- В отличие от создания нового пула на каждый HTTP-запрос (что приводило к исчерпанию оперативной памяти и зависанию ОС при множественных запросах с UI), единая очередь балансирует нагрузку между ядрами.
- Использование процессов (а не потоков) обходит ограничение GIL в Python, позволяя утилизировать 100% вычислительной мощности.

3.3. Аналитика и Парето-фронт (analytics.py)

После завершения массива симуляций модуль агрегации вычисляет интегральные метрики. Особое внимание уделяется:

- **Выявлению уязвимых спутников (Vulnerability Ranking):** Алгоритм анализирует сценарии с $k=1$ и выявляет те КА, отказ которых приводит к наибольшему (в %) падению общей доступности сети.
- **Оптимизации (Pareto Front):** Если на вход подаются различные конфигурации группировки (например, с оптимизацией RAAN), алгоритм отсеивает те, которые доминируются по доступности, стоимости и максимальному времени простоя.

Заключение

Построенная на базе метода Монте-Карло модель позволяет перевести оценку устойчивости спутниковой сети из абстрактных рассуждений в строгие математические и финансовые показатели. Техническая реализация через глобальный пул процессов и стратифицированное сэмплирование обеспечивает возможность расчёта тысяч сценариев за секунды даже на бытовых компьютерах.